KSpider:

AI Study Guide

Members:

Kirneill Joseph

Daniel Labrador

Jose Javier Lucas Casas

Goncalo F Ferreira Nascimento

Lenyn Gabriel Murillo Holguin

Knight Foundation School of Computing and Information Sciences (KFSCIS)

CIS3950 - Spring 2026

Professor Sadjadi

Table Of Contents

I. PROBLEM STATEMENT AND MOTIVATIONS……1

A. Statement……....1

B. Motivations……1

C. Target Users……...1

II. REQUIREMENTS……...1

A. System Overview……2

B. Purpose……2

C. Functional Requirements…….3

D. Non-Functional Requirements……..3

III. ARCHITECTURE……...…4

A. Description……...4

B. Structure……4

C. Design Reasoning……4

IV. DESIGN ARTIFACTS…….5

A. Entity-Relationship Diagram (ERD).....……...5

B. Ingestion Pipeline Flow……...6

C. Generation Pipeline Flow……6

D. Class / Module Relationship UML Style……...7

E. User Stories……..8

F. Use Cases……

I. PROBLEM STATEMENT AND MOTIVATIONS

A. Statement

Students often face situations where time is restrictive with their schedule and study habits, or simply look for easier ways to gather the information they are interested in. Such information might be in form of PDF, Text Files, PowerPoints (PPTX), or rarely some JSON files. This project aims at making efficient use of time, and gather interest into studying by making it simple.

- Material lives across PPTX, PDF, TXT, JSON
- Skip lengthy reading, time-consuming
- Easy export gateways

B. Motivation

The gap between "having the material" and "having structured practice materials" is a pure workflow problem, solvable with automation or extra help. Artificial Intelligence models, particularly with structured output capabilities, are now reliable enough to generate flashcards and quiz questions from source content without going out of track like they used to. Hence, this project targets such gap; take the material and produce ready to use assets!

C. Target Users

Type	Need
Students	Convert lengthy lecture PDFs or slides to save up time
Self-learners	Generate quizzes from downloaded materials
Instructors	Generate ready-to-go practice materials
Professionals	Generate structured documentation

II. REQUIREMENTS

A. System Overview

The system is an AI-powered platform that converts uploaded content into structured study materials. It supports documents and multimedia files, extracting and processing their content. The system generates outputs such as flashcards,

quizzes, and summaries using language models. It is designed with a modular backend and an optional web interface for user interaction.

B. Purpose

This document defines the functional and non-functional requirements of the system. It establishes a clear understanding of system behavior, constraints, and performance expectations. The document is intended for developers, stakeholders, and evaluators. It serves as a reference for design, implementation, and validation throughout the project.

C. Functional Requirements

1. File Ingestion
    a. System SHALL accept files in formats: .pptx, .pdf, .txt, .md
    b. System SHALL compute SHA-256 hashes for duplicate detection
    c. System SHALL extract raw text from document formats
    d. System SHALL transcribe audio/video files via OpenAI Whisper
    e. System SHALL store extracted content and metadata in persistent database
2. Text Processing

System SHALL chunk documents into semantically coherent segments

    b. Chunking SHALL prioritize paragraph/heading boundaries over arbitrary splits
    c. System SHALL fall back to fixed-size chunking with overlap when structure is absent
    d. System SHALL filter out chunks below a minimum character threshold
3. Generation
    a. System SHALL generate flashcard sets (question / answer / tags / difficulty)
    b. System SHALL generate multiple-choice quizzes with explanations
    c. System SHALL generate mixed-type practice tests (MC + T/F + short answer)
    d. System SHALL generate audio-friendly summaries for TTS consumption

e. Generation SHALL use OpenAI Structured Outputs for schema-validated responses

f. System SHALL allow user-specified item count per generation call

4. Export
    a. System SHALL export study sets to JSON (canonical format with metadata)
    b. System SHALL export flashcards to Anki-compatible CSV (semicolon-delimited)
    c. System SHALL export quizzes and tests to formatted Markdown
5. Interface
    a. System SHALL provide a FastAPI REST backend exposing all core operations
    b. System SHALL provide a Next.js frontend for file upload, browsing, and generation
    c. Web UI SHALL support drag-and-drop file upload with progress indication
    d. Web UI SHALL provide interactive flashcard viewer with flip animation
    e. Web UI SHALL provide an interactive quiz player for in-browser practice

D. Non-Functional Requirements

1. Ingestion latency
    a) Less than 5 seconds per document

2. Generation call
    a) Less than 30 seconds end to end

3. DB durability
    a) No data loss or crashes
4. Duplicate detection
    a) SHA-236 hash comparison before insert
5. Token guardrails/limit
    a) Max 5 chunks, 4000 tokens per response
6. OS support
    a) Windows, Linux, macOS

7. Python version
    a) 3.11 and 3.12
8. API key handling
    a) .evn file

III. ARCHITECTURE

A. Description

The application is structured in four layers. The separation ensures that business logic in the Application Layer has no dependency on how it is surfaced (CLI vs. Web), and that the Data Layer can evolve independently of generation strategies.

B. Structure

	AI STUDY GUIDE "KSpider"
FRONTEND	**Web UI:** File Upload, Document, Browser, Study Set Viewer, Export **CLI:** Ingest, generate, export, status
API	POST /api/ingest GET /api/documents POST /api/generate GET /api/study-sets/:id GET /api/export/:id DELETE /api/documents/:id
APP	Ingestion: ● FileScanner: discovers files in directory ● Extractors: PPTX, PDF, TXT, MD ● TextChunker: splits content to generate Generation: ● Schemas: structured output models ● Prompts: LLM prompt templates ● Generator: OpenAI API integration Export: ● JSONExporter ● MarkdownExporter ● AnkiExporter
DATABASE	sources documents chunks study_sets
EXTERNAL	OpenAI GPT-4o: Structured generation

	OpenAI Whisper: Audio, video transcription

C. Design Reasoning

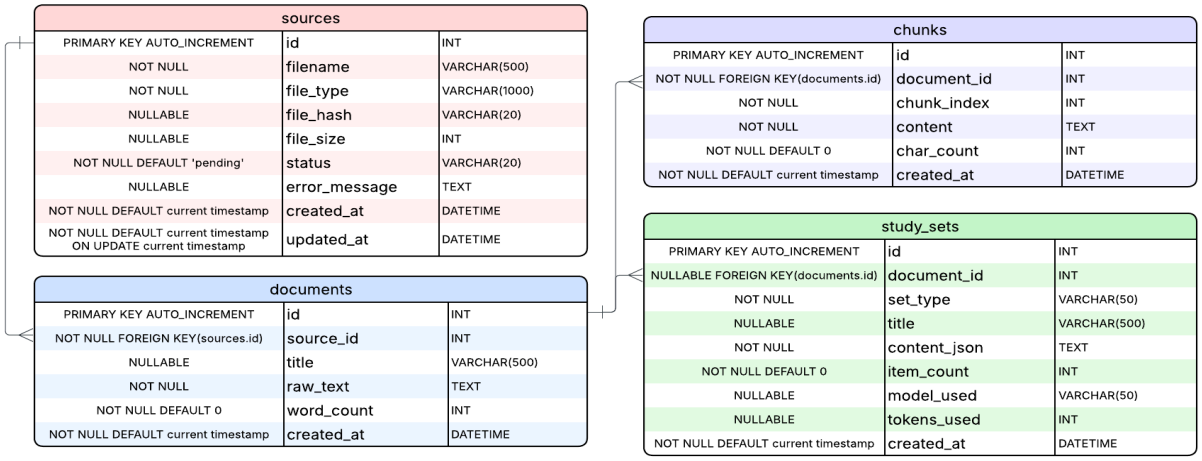
Database: SQLite	Eliminates infrastructure dependencies. The SQLAlchemy ORM abstracts the connection so future migration to PostgreSQL requires only a connection string change
Structured/Modular Outputs	OpenAI's beta.chat.completions.parse() API is used with Pydantic schemas. This guarantees schema-valid JSON responses,

	eliminating brittle regex parsing of freeform LLM output.
Chunking	Preserves coherence in generation prompts, improving quality.
Guardrails/Limits	MAX_CHUNKS_PER_GENERATION (default: 5) and MAX_TOKENS_PER_RESPONSE (default: 4,000) are environment-configurable hard limits applied before every API call. This caps per-call cost while remaining adjustable for power users.

IV. DESIGN ARTIFACTS

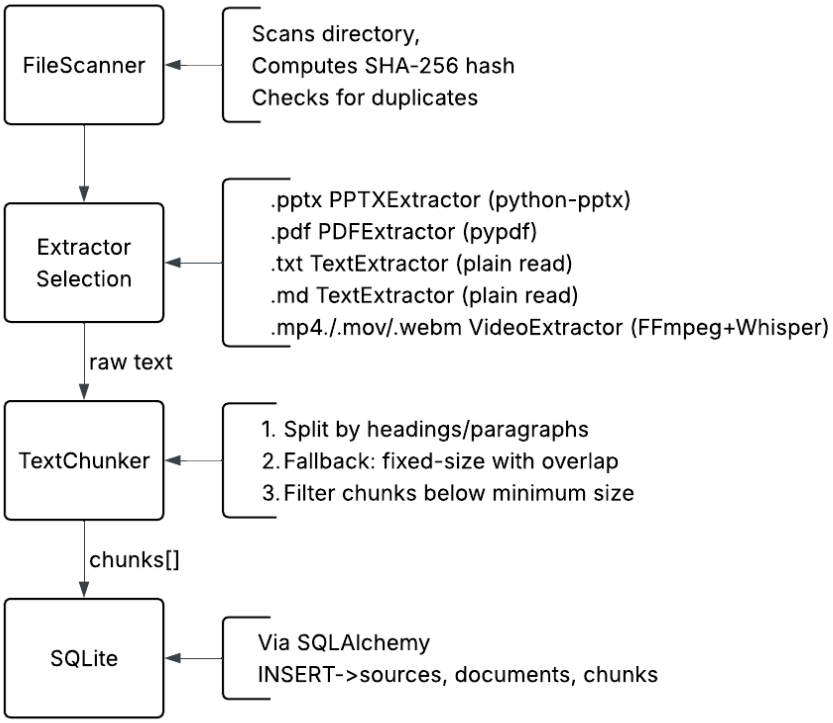
A. Entity-Relationship Diagram (ERD)

The database contains four primary entities. Sources represent raw files. Documents hold extracted text. Chunks are text segments derived from Documents and fed to the generation engine. Table study_sets store generated output as JSON blobs alongside metadata.

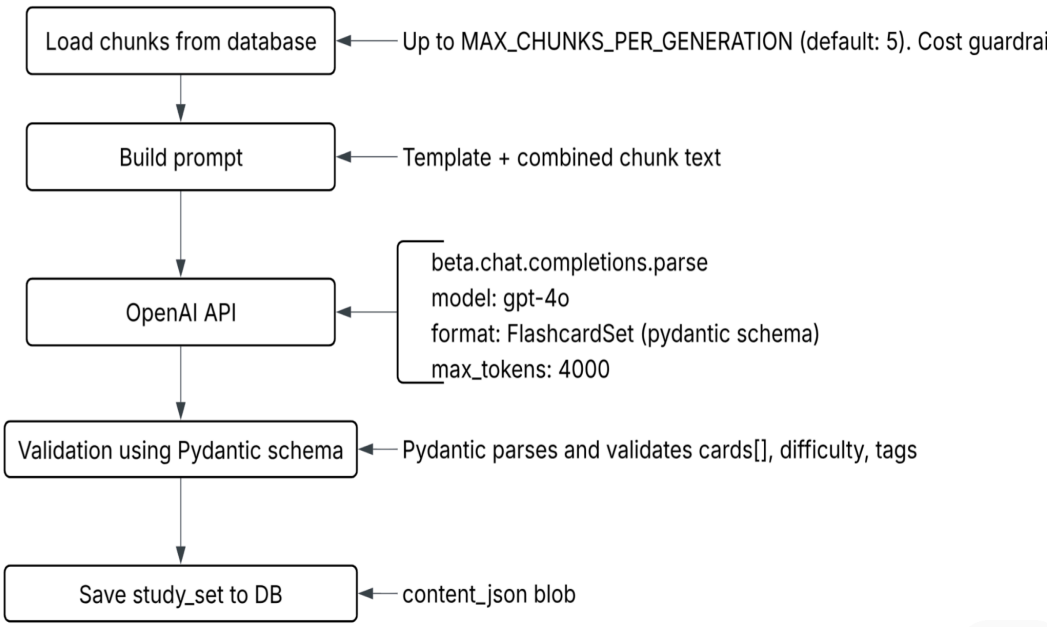


B. Ingestion Pipeline Flow

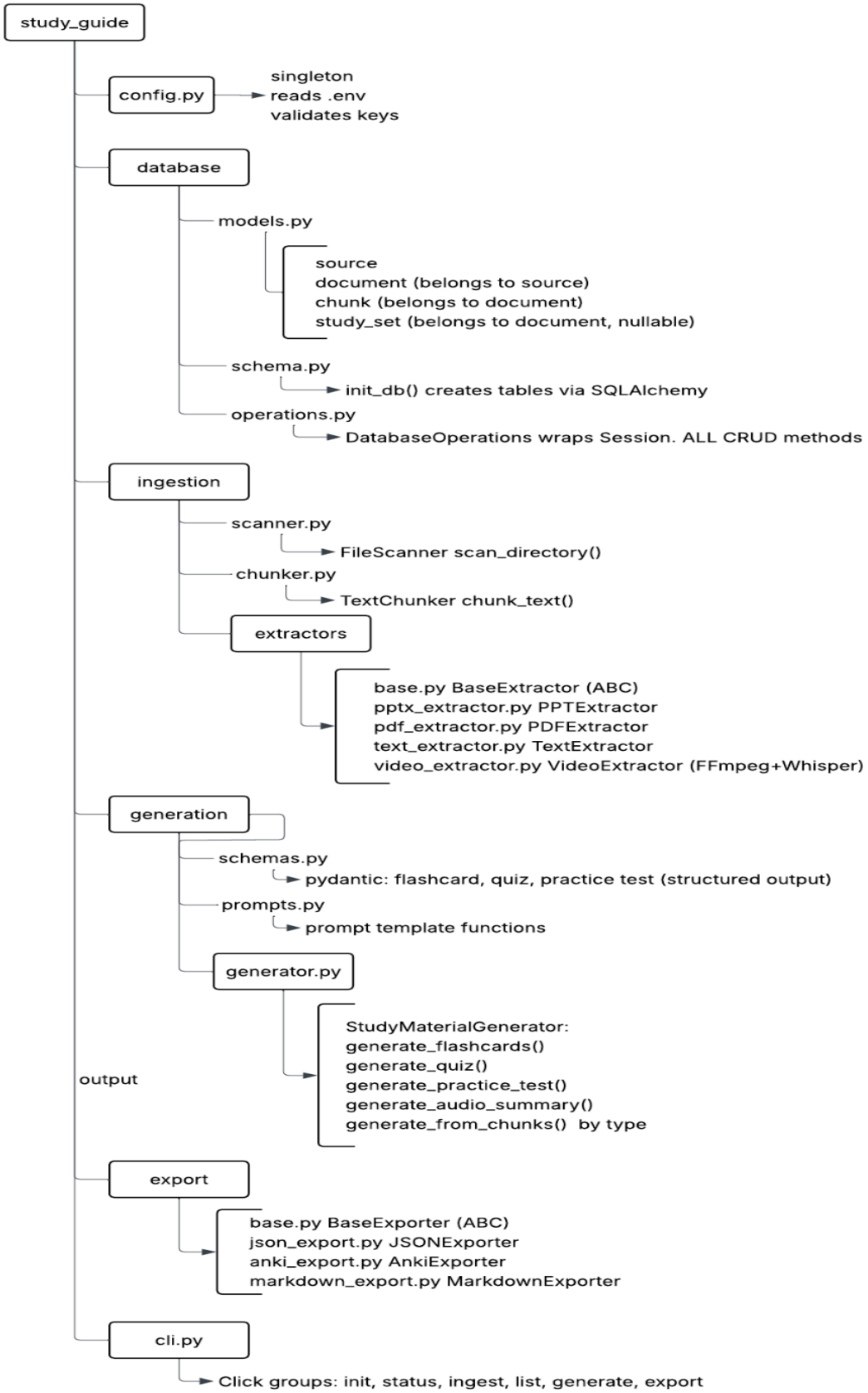
User runs: python -m study_guide ingest ./materials



C. Generation Pipeline Flow



D. Class / Module Relationship UML Style



E. User Stories

As a…	I want to…
Student	Ingest my entire lecture slide folder at once / I don't have to process files one by one
Student	Generate 30 flashcards from a PDF in one command / I can start drilling immediately
Student	Export flashcards to Anki CSV / I can use my existing spaced repetition workflow
Self-learner	Ingest a recorded lecture (MP4) and generate a quiz / I can test retention without re-watching
Instructor	Generate a practice test from my slide deck / Students have take-home practice material
Any User	See a status summary of all ingested documents / I can track what has been processed
Any user	Delete a document and its study sets / I can manage storage and remove stale content
Developer	Configure token/chunk limits via .env / I can control API costs without code changes

F. Use Cases

1. Core Ingest and Generate

Main	1. FileScanner discovers all supported files in directory 2. For each file: hash computed, duplicate check performed 3. Appropriate extractor processes file → raw text 4. TextChunker splits text into chunks → stored in DB 5. User runs: generate flashcards --doc 1 --count 20 6. Generator loads chunks (up to MAX_CHUNKS limit) 7. OpenAI API called with structured output schema 8. FlashcardSet validated by Pydantic, saved as StudySet 9. User runs: export 1 --format anki 10. AnkiExporter writes semicolon-delimited CSV to ./data/exports/

Alternate	If file is duplicate (hash match): skipped with warning, not re-processed
Exception	If OpenAI API key missing: error displayed, process aborted cleanly
Postconditions	Source, Document, Chunks, and StudySet records in DB; export file on disk

2. Video Lecture Ingestion

Name	Transcribe video and generate quiz
User type	Self-learner
Preconditions	FFmpeg installed on system; video file available
Trigger	Video file (.mp4/.mov/.webm) present in ingest directory
Main	1. VideoExtractor detects file format via extension 2. FFmpeg extracts audio track, converts to mono 64kbps MP3 3. Audio chunked into ≤25MB segments (Whisper API limit) 4. Each segment sent to OpenAI Whisper for transcription 5. Transcripts concatenated → raw_text stored in Document 6. Standard chunking and generation proceed as UC-01
Exception	If FFmpeg not installed: extractor raises clear error with install instructions
Postconditions	Full transcript stored; quiz/flashcards generatable from video content

.